

LAPORAN TUGAS KECIL 2

IF2211 STRATEGI ALGORITMA

VOXELIZATION OBJEK 3D MENGUNAKAN OCTREE



Disusun oleh:

Yusuf Faishal Listyardi

NIM: 13524014

Zahran Alvan Putra Winarko

NIM: 13524124

PROGRAM STUDI TEKNIK INFORMATIKA
SEKOLAH TEKNIK ELEKTRO DAN INFORMATIKA
INSTITUT TEKNOLOGI BANDUNG

2026

Daftar Isi

1	Dekripsi Masalah	4
2	ALGORITMA	5
2.1	Deskripsi Umum Program	5
2.2	Struktur Data Octree	5
2.2.1	Inisialisasi Root	5
2.3	Algoritma Divide and Conquer	5
2.4	Pseudocode	6
2.5	Uji Interseksi Triangle-AABB: Separating Axis Theorem	8
2.5.1	Ide Dasar Pengujian	8
2.5.2	Mengapa Menggunakan AABB?	9
2.5.3	Tiga Belas Sumbu Kandidat	9
2.5.4	Langkah Pengujian dalam Implementasi	10
2.6	Analisis Kompleksitas	10
2.6.1	Kompleksitas Waktu	10
2.6.2	Kompleksitas Ruang	11
3	IMPLEMENTASI	12
3.1	Bahasa dan Tools	12
3.2	Struktur Program	13
3.2.1	Organisasi File	13
3.3	Modul <code>Vec3.hpp</code>	13
3.4	Modul <code>AABB.hpp</code>	14
3.4.1	Fungsi <code>intersectsTriangle</code>	14
3.4.2	Fungsi <code>subdivide</code>	14
3.5	Modul <code>Octree.hpp</code>	14
3.5.1	Fungsi <code>build()</code>	14
3.5.2	Fungsi <code>buildRecursive()</code>	14
3.6	Modul <code>Viewer.hpp</code> (Bonus)	15
3.7	Modul <code>main.cpp</code>	15
3.8	Cara Kompilasi dan Menjalankan Program	16
3.8.1	Kompilasi	16

3.8.2	Menjalankan Program	16
3.8.3	Kontrol Viewer	17
3.9	Penjelasan Implementasi	17
3.10	Penjelasan Implementasi Spesifikasi Bonus	18
3.10.1	Bonus Concurrency	18
3.10.2	Bonus Viewer 3D	18
4	EKSPERIMEN	20
4.0.1	Test Case 1: <code>cow.obj</code> (depth = 4)	20
4.0.2	Test Case 2: <code>house.obj</code> (depth = 6)	21
4.0.3	Test Case 3: <code>line.obj</code> (depth = 3)	22
4.0.4	Test Case 4: <code>pumpkin.obj</code> (depth = 5)	23
4.0.5	Test Case 5: <code>teapot.obj</code> (depth = 7)	24
5	Penutup	25
5.1	Kesimpulan	25
5.2	Saran	25
5.3	Tautan	26
5.4	Lampiran	26
6	Pernyataan tidak melakukan kecurangan yang ditandatangani	27

Daftar Tabel

2.1	Analisis Kompleksitas Waktu Algoritma	11
3.1	Stack Teknologi yang Digunakan	12
3.2	Deskripsi Setiap File Source Code	13
3.3	Kontrol Viewer 3D Interaktif	17
5.1	Checklist kelengkapan program	26

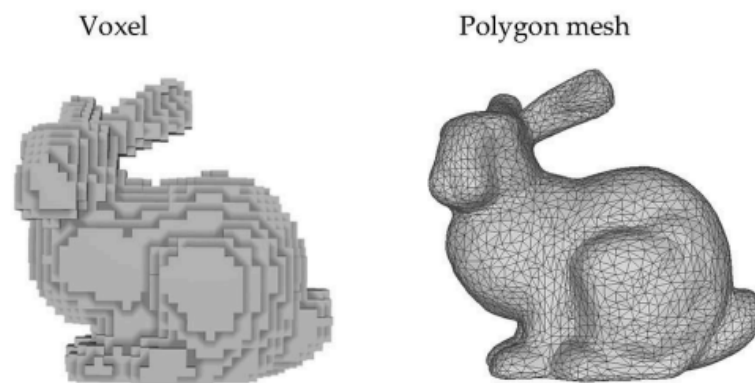
Daftar Gambar

1.1	Visualisasi voxelization	4
4.1	Input dan output CLI untuk cow.obj (depth = 4)	20
4.2	Hasil voxelization cow.obj (depth = 4)	20
4.3	Input dan output CLI untuk house.obj (depth = 6)	21
4.4	Hasil voxelization house.obj (house = 6)	21
4.5	Input dan output CLI untuk line.obj (depth = 3)	22
4.6	Hasil voxelization line.obj (depth = 3)	22
4.7	Input dan output CLI untuk pumpkin.obj (depth = 5)	23
4.8	Hasil voxelization pumpkin.obj (line = 5)	23
4.9	Input dan output CLI untuk teapot.obj (depth = 7)	24
4.10	Hasil voxelization teapot.obj (depth = 7)	24

BAB 1

Dekripsi Masalah

Representasi tiga dimensi digital pada komputer dapat dilakukan dalam berbagai format. Salah satu format yang paling umum adalah *polygon mesh*, yaitu kumpulan segitiga yang membentuk permukaan objek, umumnya disimpan dalam format `.obj`. Di sisi lain, representasi berbasis *voxel* — elemen volumetrik berbentuk kubus seperti piksel dalam ruang tiga dimensi — semakin banyak digunakan dalam berbagai aplikasi, mulai dari simulasi fisika, pencetakan 3D, visualisasi medis, hingga permainan video bergaya *blocky* seperti Minecraft.



Gambar 1.1: Visualisasi voxelization

Voxelization adalah proses mengonversi objek 3D kontinu (*polygon mesh*) menjadi representasi diskrit berbasis voxel. Tantangan utamanya adalah efisiensi: sebuah model bisa memiliki ratusan ribu segitiga, dan menguji setiap segitiga terhadap setiap calon voxel secara langsung memiliki kompleksitas $O(F \times N^3)$ di mana F adalah jumlah *face* dan N adalah resolusi tiap dimensi, yang sangat tidak praktis untuk ukuran besar.

Solusi efisien untuk masalah ini adalah menggunakan struktur data **Octree** — sebuah pohon di mana setiap node internal memiliki tepat 8 anak — dikombinasikan dengan paradigma **Divide and Conquer**. Dengan Octree, ruang tiga dimensi dibagi secara rekursif menjadi delapan oktant hingga resolusi yang diinginkan tercapai, dan hanya cabang yang bersinggungan dengan permukaan objek yang ditelusuri lebih lanjut.

BAB 2

ALGORITMA

2.1 Deskripsi Umum Program

Algoritma yang digunakan adalah **Divide and Conquer** berbasis **Octree**. Ide dasarnya sederhana: daripada memeriksa setiap segitiga terhadap setiap voxel potensial, kita memulai dari satu kotak besar yang mencakup seluruh model, kemudian membaginya secara rekursif menjadi delapan oktant yang lebih kecil. Pada setiap pembagian, kita hanya meneruskan segitiga-segitiga yang benar-benar berpotongan dengan oktant tersebut ke level berikutnya. Oktant yang tidak berisi segitiga langsung dibuang (*pruned*), sehingga komputasi terkonsentrasi hanya pada bagian ruang yang benar-benar terisi permukaan objek.

2.2 Struktur Data Octree

Octree adalah pohon (*tree*) di mana setiap node internal memiliki tepat **8 anak**. Setiap node merepresentasikan sebuah *Axis-Aligned Bounding Box* (AABB) — kotak yang sisi-sisinya sejajar dengan sumbu koordinat X, Y, dan Z.

Analogi: Bayangkan sebuah kubus besar. Jika kita belah dengan tiga bidang tegak lurus (masing-masing sejajar dengan sumbu X, Y, dan Z tepat di tengah), kita mendapatkan 8 kubus kecil berukuran setengahnya. Inilah satu langkah subdivisi Octree.

2.2.1 Inisialisasi Root

Root AABB dihitung sebagai bounding box seluruh vertex model, kemudian dibuat menjadi **kubus sempurna** menggunakan sisi terpanjang dari ketiga dimensi (ditambah faktor *padding* 0,1%). Hal ini menjamin bahwa semua voxel yang dihasilkan berukuran seragam (*uniform voxels*).

2.3 Algoritma Divide and Conquer

Algoritma bekerja dalam tiga tahap khas Divide and Conquer:

DIVIDE Program terlebih dahulu membentuk *root bounding box* yang mencakup

seluruh model 3D, lalu menyesuaikannya menjadi kubus agar ukuran voxel seragam. Setelah itu, pada setiap pemanggilan `buildRecursive()`, ruang AABB saat ini dibagi menjadi 8 sub-ruang yang lebih kecil menggunakan fungsi `subdivide()`. Karena tiap sub-ruang merupakan versi lebih kecil dari persoalan semula, maka proses voxelisasi dapat dipecah menjadi delapan upa-persoalan yang sejenis.

CONQUER Masing-masing upa-persoalan diselesaikan dengan memeriksa segitiga mana saja yang masih berpotongan dengan setiap oktan anak menggunakan uji *Triangle-AABB intersection* berbasis *Separating Axis Theorem* (SAT). Jika suatu oktan tidak lagi memiliki segitiga aktif, rekursi pada cabang tersebut dihentikan (*pruning*). Jika kedalaman maksimum telah tercapai, AABB tersebut langsung dinyatakan sebagai sebuah voxel. Jika belum, prosedur yang sama dipanggil kembali secara rekursif pada setiap oktan anak. Pada kedalaman awal, penyelesaian delapan sub-masalah ini dapat dijalankan paralel menggunakan `std::async` agar waktu komputasi lebih cepat.

COMBINE Solusi dari tiap sub-persoalan digabungkan dengan cara mengumpulkan seluruh *leaf node* valid hasil rekursi menjadi himpunan voxel akhir program. Setiap cabang yang mencapai kondisi *leaf* menambahkan AABB-nya ke dalam container `voxels`, sehingga gabungan semua hasil anak membentuk representasi voxel dari model asli. Pada bagian paralel, seluruh *thread* anak terlebih dahulu disinkronkan dengan `future::get()`, sehingga hasil dari kedelapan cabang benar-benar telah terkumpul sebelum proses kembali ke level induknya.

Kondisi basis:

- Jika daftar *face* kosong: node di-*prune*, dicatat sebagai node yang tidak perlu ditelusuri.
- Jika `depth == maxDepth`: node dicatat sebagai satu **voxel** output.

2.4 Pseudocode

Berikut adalah pseudocode fungsi inti algoritma:

```

1 FUNGSI build():
2     Hitung AABB root dari seluruh vertex model
3     Jadikan root berbentuk kubus sempurna (padding 0.1%)
4     allFaces <- [0, 1, 2, ..., F-1]
5     buildRecursive(rootAABB, allFaces, depth = 0)

```

```

6
7 FUNGSI buildRecursive(box, faces, depth):
8     // Catat statistik node yang terbentuk
9     JIKA depth > 0 MAKA
10        nodesFormed[depth]++
11
12     // BASIS 1: Tidak ada permukaan pada area ini
13     JIKA faces KOSONG MAKA
14        JIKA depth > 0 MAKA
15            nodesSkipped[depth]++
16        RETURN
17
18     // BASIS 2: Kedalaman maksimum tercapai
19     JIKA depth == maxDepth MAKA
20        tambahkan box ke daftar voxels
21        voxelCount++
22        RETURN
23
24     // DIVIDE: Bagi box saat ini menjadi 8 upa-persoalan
25     subBoxes[8] <- box.subdivide()
26     childFaces[8] <- array of list kosong
27
28     // Persiapkan data untuk tiap sub-persoalan
29     UNTUK setiap faceIdx dalam faces:
30        v0, v1, v2 <- vertex dari face[faceIdx]
31        UNTUK i dari 0 hingga 7:
32            JIKA subBoxes[i].intersectsTriangle(v0, v1, v2)
33                MAKA
34                    tambahkan faceIdx ke childFaces[i]
35
36     // CONQUER: Selesaikan masing-masing upa-persoalan
37     JIKA depth < PARALLEL_DEPTH_LIMIT MAKA
38        UNTUK i dari 0 hingga 7:
39            threads[i] <- async(buildRecursive(subBoxes[i],
40                                                childFaces[i],
41                                                depth + 1))
42
43        UNTUK i dari 0 hingga 7:
44            tunggu threads[i] selesai
45
46     JIKA TIDAK MAKA
47        UNTUK i dari 0 hingga 7:
48            buildRecursive(subBoxes[i], childFaces[i], depth +

```



```
1)
46
47 // COMBINE: Hasil dari seluruh anak tergabung dalam daftar
    voxels
48 // karena setiap leaf valid menambahkan box-nya ke
    container global
49 RETURN
50
51 AKHIR FUNGSI
```

Listing 2.1: Pseudocode Algoritma Voxelization Divide and Conquer

2.5 Uji Interseksi Triangle-AABB: Separating Axis Theorem

Untuk menentukan apakah sebuah segitiga berpotongan dengan sebuah *Axis-Aligned Bounding Box* (AABB), program ini menggunakan *Separating Axis Theorem* (SAT). Teorema ini menyatakan bahwa dua buah bangun konveks **tidak berpotongan jika dan hanya jika** terdapat minimal satu sumbu di mana proyeksi kedua bangun **tidak saling tumpang-tindih**. Sebaliknya, jika pada seluruh sumbu kandidat proyeksinya selalu saling bertumpang-tindih, maka kedua objek tersebut berpotongan.

Dalam konteks program ini, dua objek yang diuji adalah:

1. sebuah **segitiga** yang merepresentasikan salah satu face dari model 3D, dan
2. sebuah **AABB** yang merepresentasikan ruang voxel atau subruang octree yang sedang diperiksa.

Tujuan pengujian ini adalah menentukan apakah suatu face masih relevan untuk diteruskan ke suatu oktan. Jika segitiga tidak berpotongan dengan AABB suatu oktan, maka face tersebut tidak perlu diproses lebih lanjut pada cabang tersebut.

2.5.1 Ide Dasar Pengujian

Pengujian dilakukan dengan cara memproyeksikan segitiga dan AABB ke sejumlah sumbu kandidat. Pada setiap sumbu, akan diperiksa apakah interval proyeksi keduanya saling overlap. Apabila ditemukan satu saja sumbu yang menghasilkan interval tidak overlap, maka sumbu tersebut menjadi *separating axis*, sehingga segitiga dan AABB dipastikan tidak berpotongan.

Dengan kata lain:

- jika ada sumbu pemisah, hasil interseksi adalah **false**,

- jika tidak ada satu pun sumbu pemisah, hasil interseksi adalah **true**.

Karena baik segitiga maupun AABB merupakan bangun konveks, SAT dapat diterapkan secara tepat dan efisien.

2.5.2 Mengapa Menggunakan AABB?

AABB adalah *bounding box* yang sisinya selalu sejajar terhadap sumbu koordinat X, Y, dan Z. Bentuk ini dipilih karena:

1. representasinya sederhana, cukup dengan titik minimum dan maksimum,
2. pembagian ruang menjadi delapan oktan dapat dilakukan dengan mudah,
3. pengujian proyeksi terhadap sumbu X, Y, dan Z menjadi sangat efisien.

Pada implementasi, AABB dinyatakan dengan dua titik:

$$\mathbf{min} = (x_{\min}, y_{\min}, z_{\min}), \quad \mathbf{max} = (x_{\max}, y_{\max}, z_{\max})$$

Dari dua titik ini dapat dihitung:

- titik tengah AABB (*center*),
- setengah ukuran box pada tiap sumbu (*half extents*).

Segitiga kemudian ditranslasikan relatif terhadap pusat AABB agar perhitungan proyeksi menjadi lebih sederhana.

2.5.3 Tiga Belas Sumbu Kandidat

Untuk pasangan segitiga dan AABB, terdapat **13 sumbu kandidat** yang perlu diuji, yaitu:

1. **3 sumbu normal face AABB**, yaitu sumbu X, Y, dan Z.

Sumbu-sumbu ini digunakan untuk memeriksa apakah segitiga berada sepenuhnya di sebelah kiri/kanan, bawah/atas, atau depan/belakang box. Jika pada salah satu sumbu ini proyeksinya tidak overlap, maka interseksi langsung gagal.

2. **1 sumbu normal bidang segitiga**.

Sumbu ini diperoleh dari *cross product* dua sisi segitiga. Pengujian pada sumbu ini memastikan apakah AABB dan bidang segitiga saling memotong atau berada pada sisi yang terpisah.

3. **9 sumbu hasil cross product** antara 3 sumbu AABB dan 3 sisi segitiga.

Sumbu-sumbu ini diperlukan untuk menangani kasus ketika segitiga dan box

tampak overlap pada sumbu utama X, Y, Z, tetapi sebenarnya tetap terpisah jika dilihat dari arah miring tertentu. Inilah bagian penting dari SAT yang membuat pengujian menjadi akurat untuk objek 3D.

2.5.4 Langkah Pengujian dalam Implementasi

Secara umum, fungsi interseksi bekerja sebagai berikut:

1. Hitung pusat AABB dan *half extents*-nya.
2. Translasi ketiga titik segitiga agar relatif terhadap pusat AABB.
3. Hitung tiga vektor sisi segitiga.
4. Uji 9 sumbu hasil *cross product*.
5. Uji 3 sumbu utama AABB, yaitu X, Y, dan Z.
6. Uji 1 sumbu normal bidang segitiga.
7. Jika semua pengujian lolos, maka segitiga dan AABB dinyatakan berpotongan.

Setiap pengujian sumbu pada dasarnya membandingkan dua interval:

- interval proyeksi tiga titik segitiga pada sumbu tersebut,
- interval proyeksi AABB pada sumbu yang sama.

Untuk segitiga, interval diperoleh dari nilai minimum dan maksimum hasil *dot product* titik-titik segitiga terhadap sumbu uji. Untuk AABB, karena box berpusat di origin setelah translasi, jari-jari proyeksinya dapat dihitung langsung dari kombinasi *half extents* dan nilai absolut komponen sumbu.

Jika interval segitiga berada sepenuhnya di luar interval AABB, maka ditemukan sumbu pemisah dan fungsi segera mengembalikan **false**.

Implementasi pada program ini mengikuti algoritma Möller (2001), dengan optimasi kecil seperti penggunaan `std::min` dan `std::max` bersarang untuk menghindari alokasi `initializer_list` berulang. Dengan demikian, uji interseksi tetap cukup cepat meskipun dieksekusi dalam jumlah besar selama pembentukan octree.

2.6 Analisis Kompleksitas

2.6.1 Kompleksitas Waktu

- **Kasus terbaik:** $O(F)$ — semua face berada di satu oktant, tidak ada subdivisi lanjutan.
- **Kasus rata-rata:** $O(F \cdot \log_8 N)$ — distribusi normal pada permukaan objek. Di

sini $N = 8^n$ adalah jumlah voxel maksimal pada kedalaman n .

- **Kasus terburuk:** $O(F \cdot 8^n)$ — setiap face melintas semua oktant di setiap level. Tidak terjadi pada model nyata.

Tanpa optimasi, total node Octree adalah $\sum_{d=0}^n 8^d = \frac{8^{n+1}-1}{7}$. Dengan *pruning*, hanya node yang bersinggungan dengan permukaan yang dikunjungi, sehingga jumlah node aktual jauh lebih kecil.

2.6.2 Kompleksitas Ruang

$O(F + V)$ di mana F adalah jumlah face aktif dalam rekursi saat ini dan V adalah jumlah voxel yang terbentuk. *Stack* rekursi memiliki kedalaman maksimal n (parameter `maxDepth`).

Tabel 2.1: Analisis Kompleksitas Waktu Algoritma

Kasus	Kompleksitas Waktu	Keterangan
Terbaik	$O(F)$	Semua face di satu oktant
Rata-rata	$O(F \cdot \log_8 N)$	Distribusi normal permukaan
Terburuk	$O(F \cdot 8^n)$	Setiap face melintas semua oktant

BAB 3

IMPLEMENTASI

3.1 Bahasa dan Tools

Tabel 3.1: Stack Teknologi yang Digunakan

Komponen	Detail
Bahasa	C++17
Build System	CMake 3.14+ / Makefile
Concurrency	<code>std::async</code> , <code>std::future</code> , <code>std::mutex</code> (C++ Standard Library)
Viewer	OpenGL 1.x (Legacy), GLFW3

3.2 Struktur Program

3.2.1 Organisasi File

Tabel 3.2: Deskripsi Setiap File Source Code

File	Deskripsi
Vec3.hpp	Struct vektor 3D dengan operasi dot, cross, normalize, dan operator overloading (+, -, *). Menggunakan constexpr dan [[nodiscard]].
AABB.hpp	Struct <i>Axis-Aligned Bounding Box</i> : fungsi center(), halfExtents(), intersectsTriangle() (SAT Möller), dan subdivide() ke 8 oktant.
ObjModel.hpp	Container sederhana untuk daftar <i>vertex</i> dan <i>face</i> hasil parsing.
ObjParser.hpp	Namespace ObjParser: baca file .obj, tangani format v/vt/vn, fan triangulation untuk poligon > 3 vertex, validasi indeks.
Octree.hpp	Kelas utama: algoritma D&C, std::async concurrency, pengumpulan voxel, dan statistik <i>nodesFormed/nodesSkipped</i> .
ObjWriter.hpp	Namespace ObjWriter: tulis voxel (AABB) ke file .obj — 8 vertex + 12 segitiga per voxel.
Viewer.hpp	Viewer 3D OpenGL/GLFW: orbit camera, render solid + wireframe, toggle model asli (efek X-Ray), implementasi manual <i>LookAt</i> tanpa GLU.
main.cpp	Entry point: parsing argumen CLI, orkestrasi pipeline, output statistik.

3.3 Modul Vec3.hpp

Struct Vec3 menyimpan koordinat (x, y, z) bertipe double dan menyediakan operasi-operasi dasar vektor 3D yang dibutuhkan oleh AABB dan renderer:

- operator+, operator-, operator* — operasi aritmetika per-komponen.
- dot() — hasil kali titik, digunakan pada uji SAT.
- cross() — hasil kali silang, digunakan untuk menghitung normal bidang segitiga dan sumbu LookAt.
- normalize() — normalisasi dengan pengecekan panjang > 0.
- operator« — overload untuk output stream langsung.

Seluruh operasi aritmetika dideklarasikan `constexpr` sehingga dapat dievaluasi pada *compile time* jika operand diketahui.

3.4 Modul `AABB.hpp`

3.4.1 Fungsi `intersectsTriangle`

Implementasi SAT sesuai Möller (2001) yang sudah dioptimasi. Menggunakan `std::min/std::max` bersarang (bukan `initializer_list`) untuk menghindari alokasi heap berulang pada loop rekursif yang intensif.

3.4.2 Fungsi `subdivide`

Mengembalikan `std::array<AABB, 8>` (bukan `vector`) sehingga 8 oktant anak dialokasikan di *stack* tanpa overhead heap.

3.5 Modul `Octree.hpp`

Kelas `Octree` adalah inti program. Atribut utamanya:

- `model` — referensi ke model yang di-parse.
- `maxDepth` — kedalaman rekursi maksimum.
- `voxels` — vektor `AABB` hasil akhir.
- `stats` — struct berisi `nodesFormed[]`, `nodesSkipped[]`, dan `voxelCount`.
- `voxelMutex`, `statsMutex` — mutex untuk *thread-safe access* saat concurrency aktif.

3.5.1 Fungsi `build()`

Menghitung bounding box seluruh model, membuat root `AABB` menjadi kubus sempurna, mengisi `allFaces` menggunakan `std::iota`, dan memulai rekursi.

3.5.2 Fungsi `buildRecursive()`

Implementasi inti D&C yang dijelaskan pada pseudocode di Bab 2. Kunci desainnya adalah penggunaan `std::async` dengan `std::launch::async` hanya ketika `depth < PARALLEL_DEPTH_LIMIT` ($= 2$). Pada dua level teratas ini, hingga $8^2 = 64$ thread dapat berjalan paralel, memberikan speedup nyata pada mesin multi-core.

```
1 if (currentDepth < PARALLEL_DEPTH_LIMIT) {  
2     std::array<std::future<void>, 8> threads;
```

```

3   for (int i = 0; i < 8; ++i) {
4       threads[i] = std::async(std::launch::async,
5                               [this, &subBoxes, &facesPerOktant, currentDepth, i
6                                   ]() {
7                               buildRecursive(subBoxes[i],
8                                               facesPerOktant[i],
9                                               currentDepth + 1);
10                          });
11   }
12   // Barrier: tunggu semua thread selesai
13   for (auto& t : threads) t.get();
14 } else {
15     for (int i = 0; i < 8; ++i)
16         buildRecursive(subBoxes[i], facesPerOktant[i],
17                         currentDepth + 1);
18 }

```

Listing 3.1: Potongan implementasi concurrency

3.6 Modul Viewer.hpp (Bonus)

Viewer dikompilasi secara kondisional dengan flag `-DENABLE_VIEWER`. Desain utamanya:

- **Orbit camera:** State kamera disimpan dalam struct `ViewerState` yang diakses melalui pointer global (diperlukan karena callback GLFW bertipe function pointer C).
- **Manual LookAt:** Fungsi `applyLookAtMatrix()` diimplementasikan tanpa bergantung pada GLU agar kompatibel dengan Windows/MinGW yang sering tidak menyertakan GLU.
- **Efek X-Ray:** Saat toggle `T` aktif, model asli digambar dua kali — sekali sebelum voxel (tertambun) dan sekali sesudah dengan `GL_DEPTH_TEST` dimatikan — menghasilkan efek *tembus pandang*.
- **Reset otomatis:** `g_state = ViewerState{}` mereset kamera setiap kali viewer dibuka.

3.7 Modul main.cpp

Entry point memisahkan logika parsing argumen ke fungsi `parseArguments()` yang mengembalikan struct `AppConfig`, sehingga fungsi `main()` hanya berperan sebagai

orkestrator pipeline yang mudah dibaca.

3.8 Cara Kompilasi dan Menjalankan Program

3.8.1 Kompilasi

```
# Linux/Mac -- Tanpa viewer
mkdir -p bin
g++ -std=c++17 -O2 -o bin/voxelizer src/main.cpp -lpthread

# Linux -- Dengan viewer (GLFW harus terinstal)
g++ -std=c++17 -O2 -DENABLE_VIEWER -o bin/voxelizer_viewer \
    src/main.cpp -lpthread -lGL -lGLU -lglfw

# Windows (MSYS2/MinGW) -- Tanpa viewer
g++ -std=c++17 -O2 -o bin/voxelizer.exe src/main.cpp -lpthread
```

Listing 3.2: Cara kompilasi program

3.8.2 Menjalankan Program

```
# Tanpa viewer
./bin/voxelizer <path_file.obj> <max_depth>

# Dengan viewer
./bin/voxelizer_viewer <path_file.obj> <max_depth> --view

# Contoh
./bin/voxelizer test/input/cube.obj 3
./bin/voxelizer_viewer test/input/pumpkin.obj 6 --view
```

Listing 3.3: Cara menjalankan program

3.8.3 Kontrol Viewer

Tabel 3.3: Kontrol Viewer 3D Interaktif

Input	Aksi
Drag kiri mouse	Rotasi kamera (orbit)
Scroll	Zoom in/out
Tombol T	Toggle tampilkan model asli (efek X-Ray)
Q / ESC	Keluar viewer

3.9 Penjelasan Implementasi

Implementasi utama program terletak pada kelas `Octree`, khususnya pada method `build()` dan `buildRecursive()`. Kedua fungsi ini berperan dalam membangun struktur octree dan menghasilkan voxel dari model 3D masukan.

Proses dimulai pada method `build()`, yaitu dengan menghitung *bounding box* yang mencakup seluruh vertex model. Selanjutnya, *bounding box* tersebut diubah menjadi kubus sempurna agar seluruh voxel yang dihasilkan memiliki ukuran yang seragam. Setelah itu, program menyusun daftar seluruh face pada model dan memulai proses rekursif dari node akar dengan memanggil `buildRecursive()`.

Method `buildRecursive()` merepresentasikan inti algoritma *Divide and Conquer*. Pada setiap pemanggilan, fungsi menerima sebuah AABB, himpunan face aktif yang masih berpotongan dengan area tersebut, serta kedalaman saat ini. Jika himpunan face kosong, maka cabang tersebut langsung dihentikan karena tidak mengandung bagian permukaan objek. Jika kedalaman maksimum telah tercapai, AABB tersebut langsung dicatat sebagai sebuah voxel.

Jika kedua kondisi tersebut belum terpenuhi, maka AABB saat ini dibagi menjadi 8 oktan menggunakan fungsi `subdivide()`. Selanjutnya, setiap face aktif diperiksa terhadap masing-masing oktan anak. Face yang berpotongan dengan suatu oktan akan diteruskan ke node anak tersebut. Dengan cara ini, setiap submasalah hanya memproses face yang relevan dengan wilayah ruangnya.

Proses rekursif ini terus berlanjut hingga seluruh cabang mencapai kondisi berhenti. Hasil akhirnya diperoleh dengan mengumpulkan seluruh *leaf node* yang valid ke dalam daftar `voxels`. Daftar inilah yang kemudian menjadi representasi voxel dari model 3D awal.

Secara keseluruhan, implementasi ini memanfaatkan struktur octree untuk membagi

ruang secara bertahap dan hanya memproses bagian ruang yang berkaitan dengan permukaan objek, sehingga proses voxelisasi menjadi lebih efisien.

3.10 Penjelasan Implementasi Spesifikasi Bonus

3.10.1 Bonus Concurrency

Implementasi *concurrency* terdapat pada kelas `Octree`, tepatnya di dalam method `buildRecursive()`. Pada proses pembentukan octree, setiap node yang belum mencapai kondisi basis akan dibagi menjadi 8 oktan. Delapan oktan ini merupakan submasalah yang bersifat independen, sehingga dapat diproses secara bersamaan.

Pada implementasinya, ketika kedalaman rekursi masih lebih kecil dari `PARALLEL_DEPTH_LIMIT`, pemrosesan kedelapan anak dijalankan secara paralel menggunakan `std::async`. Setiap pemanggilan `buildRecursive()` untuk satu oktan anak dibungkus dalam sebuah *task* asynchronous, lalu hasilnya disimpan dalam `std::future`. Setelah seluruh *task* dijalankan, program menunggu semuanya selesai dengan memanggil `get()` pada setiap *future*.

Pendekatan ini dipilih karena pada level atas octree jumlah face yang masih aktif umumnya besar, sehingga pemrosesan paralel dapat mempercepat waktu eksekusi. Namun, paralelisme tidak diterapkan pada seluruh level. Program membatasi concurrency hanya sampai kedalaman tertentu agar tidak terjadi *thread explosion* dan overhead yang terlalu besar. Setelah melewati batas tersebut, rekursi dilanjutkan secara sekuensial.

Karena daftar voxel dan statistik dapat diakses oleh beberapa *thread* secara bersamaan, implementasi juga menggunakan `mutex` untuk menjaga konsistensi data ketika menambahkan voxel ke container hasil maupun saat memperbarui statistik node.

3.10.2 Bonus Viewer 3D

Implementasi *viewer* 3D terdapat pada file `Viewer.hpp` dan diaktifkan melalui target `voxelizer_viewer`. Setelah proses voxelisasi selesai, program dapat menampilkan model hasil dalam jendela interaktif apabila dijalankan dengan flag `-view`.

Viewer dibangun menggunakan OpenGL dan GLFW. Data utama yang ditampilkan adalah kumpulan voxel hasil konversi, yaitu daftar `AABB` yang dihasilkan oleh octree. Setiap voxel divisualisasikan sebagai kubus 3D dengan memanfaatkan fungsi `drawCube()`, sehingga pengguna dapat melihat bentuk model hasil voxelisasi secara langsung.

Selain voxel, viewer juga dapat menampilkan model asli secara transparan sebagai pembanding. Fitur ini membantu pengguna melihat seberapa baik hasil voxelisasi merepresentasikan bentuk objek awal. Pada implementasi, viewer menerima baik daftar

voxel maupun data model asli, lalu merender keduanya dalam satu scene 3D.

Interaksi pengguna juga disediakan agar tampilan tidak bersifat statis. Kamera dapat diputar dengan mekanisme *orbit*, zoom dapat diatur menggunakan *scroll mouse*, dan beberapa mode tampilan dapat diubah melalui input keyboard, seperti menampilkan atau menyembunyikan model asli serta mengganti mode *wireframe*. Dengan demikian, hasil voxelisasi tidak hanya disimpan ke file *.obj*, tetapi juga dapat diamati secara visual dan interaktif langsung dari program.

BAB 4

EKSPERIMEN

4.0.1 Test Case 1: cow.obj (depth = 4)

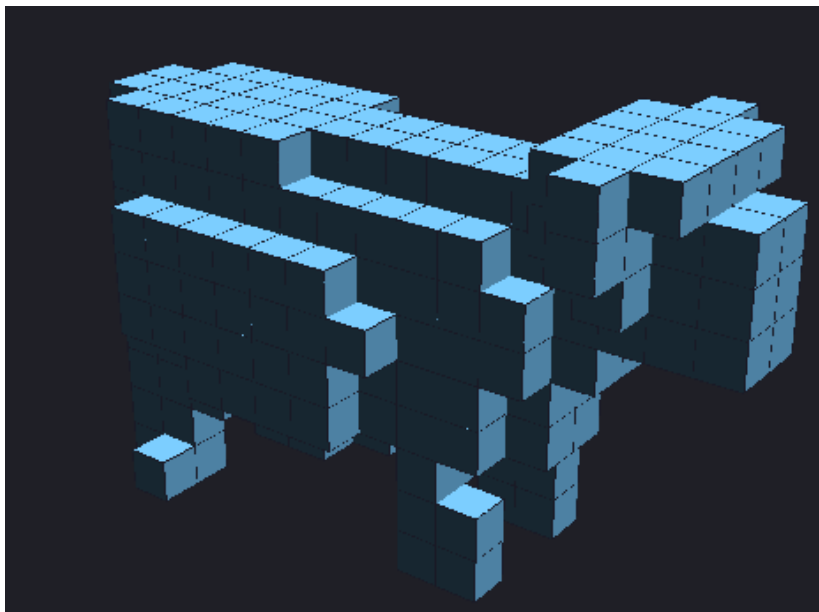
```
=== Hasil Konversi ===
Banyak voxel terbentuk : 332
Banyak vertex terbentuk : 2656
Banyak faces terbentuk : 3984

Statistik node octree yang terbentuk:
1 : 8
2 : 64
3 : 160
4 : 704

Statistik node yang tidak perlu ditelusuri:
1 : 0
2 : 44
3 : 72
4 : 372

Kedalaman octree : 4
Waktu berjalan : 12.4335 ms
Output disimpan di: /mnt/d/ITB/sem 4/STIMA/Tucil2_13524014_13524124/test/output/cow_voxel_4.obj
=====
```

Gambar 4.1: Input dan output CLI untuk cow.obj (depth = 4)

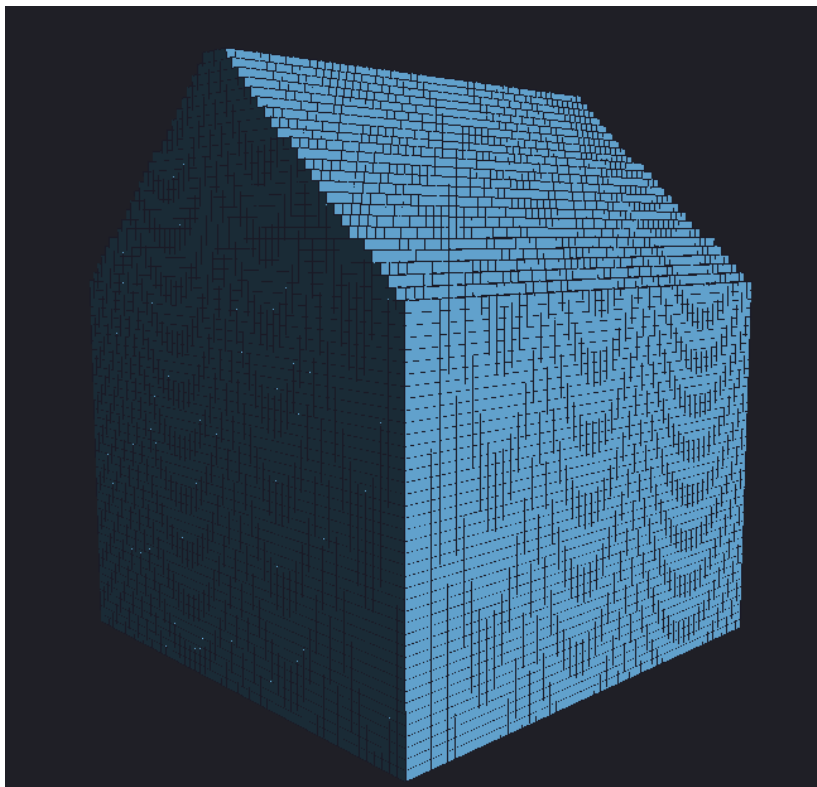


Gambar 4.2: Hasil voxelization cow.obj (depth = 4)

4.0.2 Test Case 2: house.obj (depth = 6)

```
=== Hasil Konversi ===  
Banyak voxel terbentuk : 17764  
Banyak vertex terbentuk : 142112  
Banyak faces terbentuk : 213168  
  
Statistik node octree yang terbentuk:  
1 : 8  
2 : 64  
3 : 448  
4 : 2272  
5 : 8480  
6 : 36064  
  
Statistik node yang tidak perlu ditelusuri:  
1 : 0  
2 : 8  
3 : 164  
4 : 1212  
5 : 3972  
6 : 18300  
  
Kedalaman octree : 6  
Waktu berjalan : 22.0291 ms  
Output disimpan di: /mnt/d/ITB/sem 4/STIMA/Tucil2_13524014_13524124/test/output/house_voxel_6.obj  
=====
```

Gambar 4.3: Input dan output CLI untuk house.obj (depth = 6)

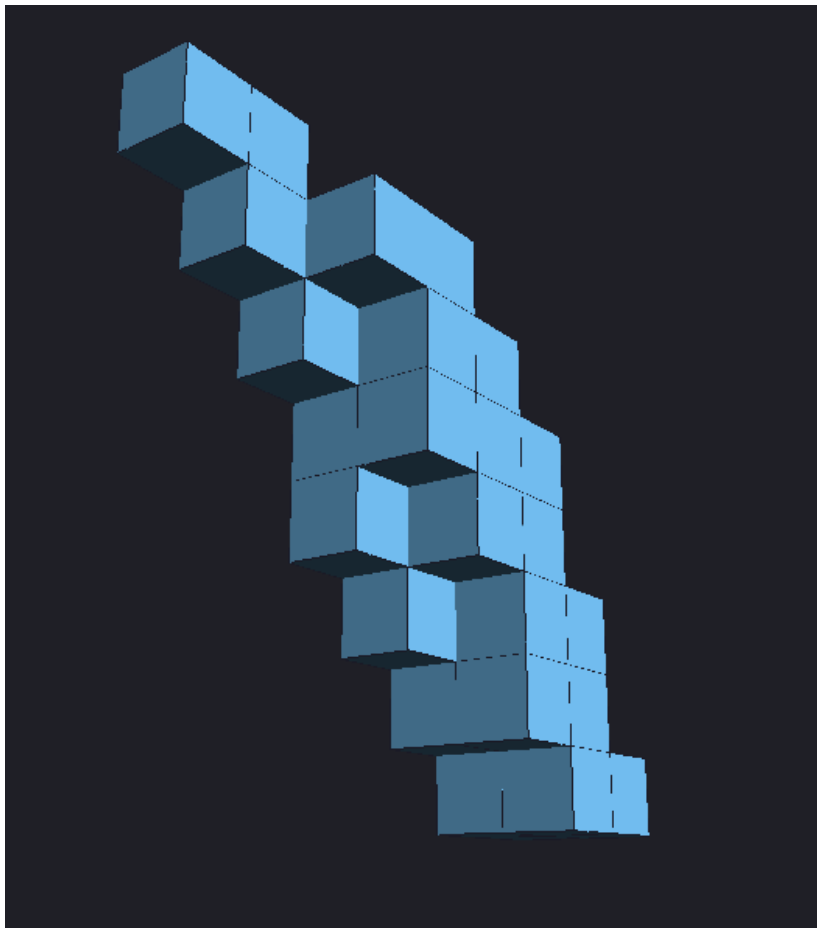


Gambar 4.4: Hasil voxelization house.obj (house = 6)

4.0.3 Test Case 3: line.obj (depth = 3)

```
=== Hasil Konversi ===  
Banyak voxel terbentuk : 33  
Banyak vertex terbentuk : 264  
Banyak faces terbentuk : 396  
  
Statistik node octree yang terbentuk:  
1 : 8  
2 : 56  
3 : 120  
  
Statistik node yang tidak perlu ditelusuri:  
1 : 1  
2 : 41  
3 : 87  
  
Kedalaman octree : 3  
Waktu berjalan : 11.0832 ms  
Output disimpan di: /mnt/d/ITB/sem 4/STIMA/Tucil2_13524014_13524124/test/output/line_voxel_3.obj  
=====
```

Gambar 4.5: Input dan output CLI untuk line.obj (depth = 3)

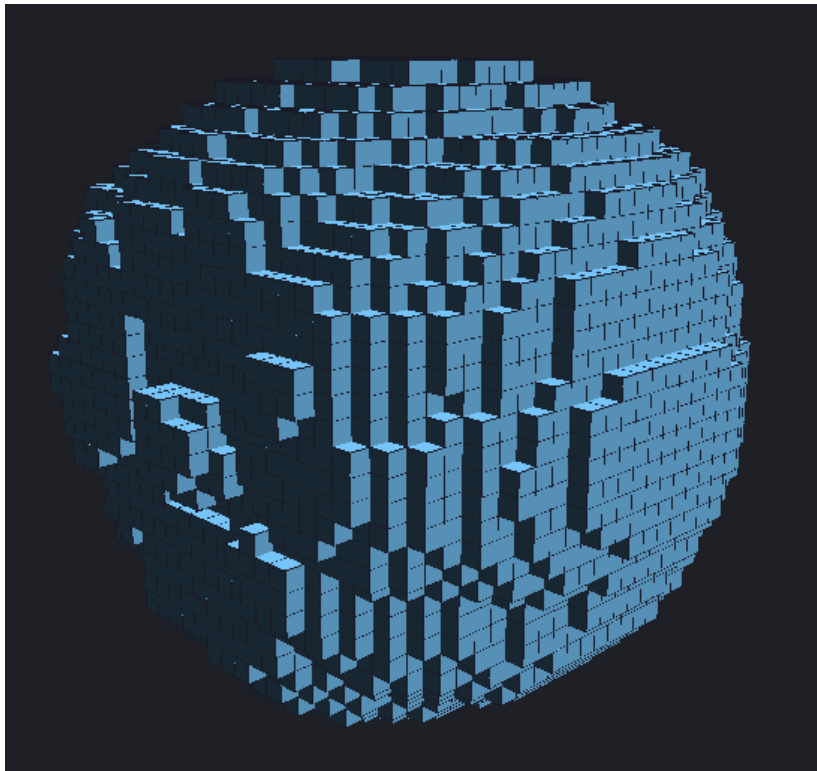


Gambar 4.6: Hasil voxelization line.obj (depth = 3)

4.0.4 Test Case 4: pumpkin.obj (depth = 5)

```
=== Hasil Konversi ===  
Banyak voxel terbentuk : 4619  
Banyak vertex terbentuk : 36952  
Banyak faces terbentuk : 55428  
  
Statistik node octree yang terbentuk:  
1 : 8  
2 : 64  
3 : 448  
4 : 2216  
5 : 8968  
  
Statistik node yang tidak perlu ditelusuri:  
1 : 0  
2 : 8  
3 : 171  
4 : 1095  
5 : 4349  
  
Kedalaman octree : 5  
Waktu berjalan : 22.5405 ms  
Output disimpan di: /mnt/d/ITB/sem 4/STIMA/Tucil2_13524014_13524124/test/output/pumpkin_voxel_5.obj  
=====
```

Gambar 4.7: Input dan output CLI untuk pumpkin.obj (depth = 5)

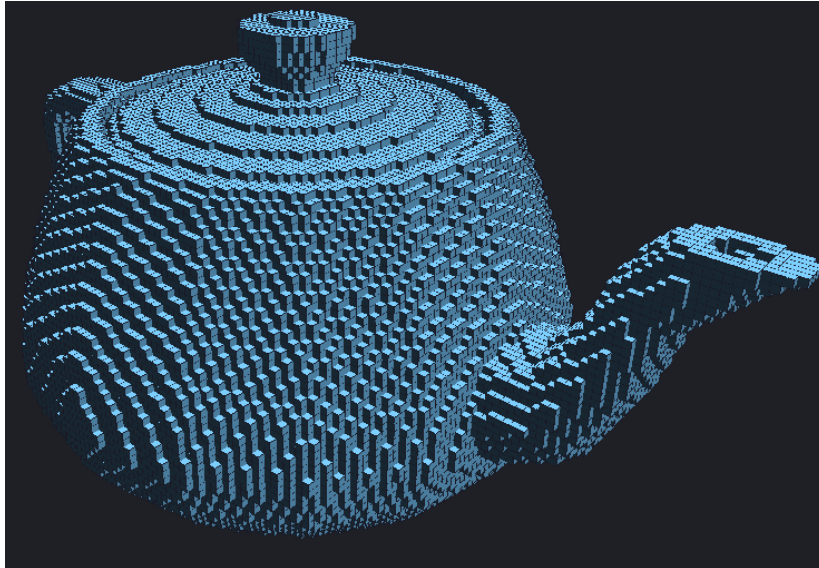


Gambar 4.8: Hasil voxelization pumpkin.obj (line = 5)

4.0.5 Test Case 5: teapot.obj (depth = 7)

```
=== Hasil Konversi ===  
Banyak voxel terbentuk : 29536  
Banyak vertex terbentuk : 236288  
Banyak faces terbentuk : 354432  
  
Statistik node octree yang terbentuk:  
1 : 8  
2 : 64  
3 : 192  
4 : 864  
5 : 3344  
6 : 13968  
7 : 58368  
  
Statistik node yang tidak perlu ditelusuri:  
1 : 0  
2 : 40  
3 : 84  
4 : 446  
5 : 1598  
6 : 6672  
7 : 28832  
  
Kedalaman octree : 7  
Waktu berjalan : 35.4388 ms  
Output disimpan di: /mnt/d/ITB/sem 4/STIMA/Tucil2_13524014_13524124/test/output/teapot_voxel_7.obj  
=====
```

Gambar 4.9: Input dan output CLI untuk teapot.obj (depth = 7)



Gambar 4.10: Hasil voxelization teapot.obj (depth = 7)

BAB 5

Penutup

5.1 Kesimpulan

Dari pengerjaan Tugas Kecil 2 ini, dapat diambil beberapa kesimpulan yaitu, program voxelization 3D berhasil diimplementasikan menggunakan algoritma Divide and Conquer berbasis Octree dalam bahasa C++17. Algoritma bekerja dengan membagi ruang secara rekursif menjadi 8 oktant, memangkas oktant yang tidak berisi permukaan (*pruning*), dan menghasilkan voxel pada leaf node yang bersinggungan dengan segitiga model. Implementasi terbukti menghasilkan output yang benar dan seragam (*uniform voxels*).

Pruning berdasarkan hasil uji SAT secara signifikan mengurangi jumlah node yang dikunjungi dibandingkan traversal penuh. Pada model dengan permukaan tipis (*thin shell*), hanya sebagian kecil dari 8^n node maksimal yang benar-benar diproses.

Selain itu Penggunaan `std::async` pada dua level kedalaman teratas Octree memberikan speedup nyata pada mesin multi-core tanpa memerlukan library eksternal. Batas `PARALLEL_DEPTH_LIMIT = 2` mencegah *thread explosion* di kedalaman bawah.

Viewer interaktif berbasis OpenGL/GLFW memudahkan verifikasi visual hasil voxelization. Implementasi manual *LookAt* tanpa GLU meningkatkan portabilitas di berbagai platform.

5.2 Saran

Pelaksanaan Tugas Kecil 2 IF2211 Strategi Algoritma di Semester II (Genap) Tahun 2024/2025 merupakan pengalaman yang sangat berharga bagi kami. Dari pengalaman ini, kami ingin berbagi beberapa saran kepada pembaca yang mungkin akan menghadapi tugas serupa di masa depan:

Ucup: Terimakasih asisten yang sudah memberi tucil di tengah libur lebaran.

Alvan: Saya pikir tubes ini didesain kepada kita untuk melatih diri menghadapi tekanan deadline yang mungkin nanti ketika di industri akan relate. Namun, tugas ini sebaiknya dikerjakan sesegera mungkin. Dengan waktu kurang lebih 2 minggu

saya merasa cukup chaos menghadapi tugas-tugas yang ada. Rasa-rasanya setelah tugas ini selesai pun ketenangan tidak serta merta menghampiri, terpaan deadline selalu menghantui. Pagi, siang malam, dilanda ketakutan akan tugas. Bingung memikirkan bagaimana ku menghadapinya. Namun, aku memiliki solusi dan saran kepadamu, “Sebaiknya kalo ada tugas minimal di-extend 5 hari”.

5.3 Tautan

Seluruh implementasi kode program tersedia di repository GitHub:

Repository: https://github.com/ucupsss/Tucil2_13524014_13524124

5.4 Lampiran

Berikut adalah tabel checklist pemenuhan spesifikasi tugas:

No	Poin	Ya	Tidak
1	Program berhasil dikompilasi tanpa kesalahan	✓	
2	Program berhasil dijalankan	✓	
3	Hasil konversi <code>.obj</code> terdiri dari kubus-kubus kecil	✓	
4	Program menampilkan informasi/ <i>report</i> di CLI	✓	
5	Program menerima masukan <code>.obj</code> dan mengembalikan output	✓	
6	[Bonus] Program menggunakan <i>concurrency</i>	✓	
7	[Bonus] Program memiliki <code>.obj</code> viewer	✓	

Tabel 5.1: Checklist kelengkapan program

BAB 6

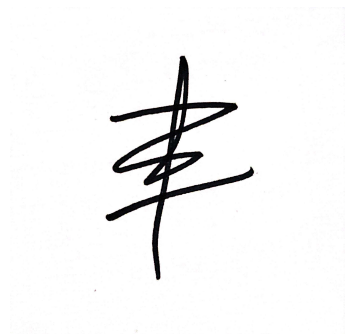
Pernyataan tidak melakukan kecurangan yang ditan- datangani

PERNYATAAN

Kami yang bertanda tangan di bawah ini menyatakan bahwa Tugas ini disusun sepenuhnya tanpa bantuan kecerdasan buatan (Generative AI), melainkan hasil pemikiran dan analisis mandiri.

A handwritten signature in black ink on a dotted grid background. The signature appears to be 'Yusuf Faishal Listyardi' written in a cursive style.

Yusuf Faishal Listyardi

A handwritten signature in black ink on a dotted grid background. The signature is stylized and appears to be 'Zahran Alvan Putra Winarko'.

Zahran Alvan Putra Winarko